

پروژه درس ساختار و زبان کامپیوتر

پروژه : طراحی برنامه Convolution

قسمت 1 : فایل های برای این بخش:

conv c.c / conv asm.c / conv total.c / conv.asm

در این بخش یک ماتریس (kernel) 3 در 3 را به عنوان ورودی می‌گیریم و بعد با این ماتریس عملیات convolution را برای یک تصویر دلخواه انجام می‌دهیم. در conv_c.c تمام فرآیند با زبان سطح بالا C انجام می‌شود. در conv_asm بخش محاسبه عملیات convolution با استفاده از تابع conv3x3 که در conv.asm تعریف شده است انجام می‌شود. در conv_total هر دو روش پردازش که به صورت جداگانه در conv_c و conv_asm پیاده‌سازی شد 20 بار پشت سرهم انجام می‌شود و در انتها زمان مصرف شده برای هر کدام از روش ها را با هم مقایسه می‌کنیم. ابتدا conv_asm را بررسی می‌کنیم:

ورودی تابع conv3x3 مانند شکل می‌باشد.

```
; rdi = input image pointer (u8)
; rsi = output image pointer (u8)
; rdx = width
; rcx = height
; r8 = kernel pointer (9 * int32s)
conv3x3:
```

و اینگونه در فایل های به زبان C استفاده می‌شود :

```
extern void conv3x3(const uint8_t* in, uint8_t* out, int width, int height, const int* kernel);
```

در ابتدا مقادیر در رجیستر های Callee-saved در پشته push می‌شوند و در انتها همگی pop می‌شوند و مقادیر اولیه آنها بازگردانده می‌شود.

برنامه به 3 بخش اصلی تقسیم می‌شود:

1. ذخیره کردن مقادیر در ماتریس ورودی در ثبات‌ها در این بخش انجام می‌شود.

ابتدا مقادیر در خانه‌های ماتریس 3 در 3 را در ثبات‌های xmm8 , ... , xmm0 قرار داده می‌شوند که در نتیجه بتوان در پردازش‌هایی که می‌خواهیم انجام بدهیم ؛ بتوانیم عملیات‌های برداری انجام دهیم که در نتیجه سرعت

انجام پردازش زیاد خواهد شد. (به جای این که عملیات های پردازش را روی یک پیکسل انجام دهیم ، همزمان روی 16 پیکسل انجام دهیم)

```
LOAD_K xmm0, 0
LOAD_K xmm1, 4
LOAD_K xmm2, 8
LOAD_K xmm3, 12
LOAD_K xmm4, 16
LOAD_K xmm5, 20
LOAD_K xmm6, 24
LOAD_K xmm7, 28
LOAD_K xmm8, 32
```

```
%macro LOAD_K 2
    mov eax, dword [r8 + %2]
    movd %1, eax
    pshufw %1, %1, 0
    punpcklqdq %1, %1
%endmacro
```

با استفاده از ماکرو **LOAD_K** انجام می شود. (مقادیر در خانه های ماتریس 4 بایت هستند).

سپس در ثبات **ebp** مقدار 1 ذخیره می شود. (این ثبات همواره نشان دهنده سطری که الان می خواهیم روی آن عملیات انجام بدهیم، می باشد) سطر ابتدایی در نظر گرفته نمی شود چرا که برای هر پیکسل در این سطر ، 8 پیکسل در همسایگی آن وجود ندارد و نمی توان عملیات **convolution** را برای این پیکسل ها انجام داد.

سپس با استفاده از 3 ثبات **r10, r11, r12** ، سطری که می خواهیم روی آن عملیات انجام دهیم همراه با سطر قبلی و بعدی این سطر را در نظر می گیریم. ثبات **r13** هم برای خروجی در نظر گرفته می شود. در ثبات **ebx** هم مقدار 1 ذخیره می شود و نقش این ثبات مانند ثبات **ebp** می باشد با این تفاوت که به جای سطر ، برای ستونی که می خواهیم روی آن عملیات انجام دهیم ، می باشد.

```
mov    eax, r9d
dec    eax
cmp    ebp, eax
jge    done

; row_offset_bytes_in =
mov    eax, ebp
imul   eax, edx
movsxd rax, eax

; cur/prev/next pointers
lea    r10, [rdi + rax]
mov    r11, r10
sub    r11, rdx
```

```
sub    r12, rax
lea    r12, [r10 + rdx]

lea    r13, [rsi + rax]

mov    ebx, 1

; if w-17 < 0 -> no 16-l
mov    eax, edx
sub    eax, 17
js     scalar_row
mov    r14d, eax
```

2. عملیات ضرب برداری در این بخش انجام می‌شود.

```
; prev row
MULL1 r11, -1, xmm0
MULL1 r11, 0, xmm1
MULL1 r11, 1, xmm2
; cur row
MULL1 r10, -1, xmm3
MULL1 r10, 0, xmm4
MULL1 r10, 1, xmm5
; next row
MULL1 r12, -1, xmm6
MULL1 r12, 0, xmm7
MULL1 r12, 1, xmm8
```

```
%macro MULL1 3
    movdqu xmm12, [%1 + rbx + %2]

    movaps xmm13, xmm12

    punpcklbw xmm13, xmm9
    pmullw    xmm13, %3
    paddw     xmm10, xmm13

    punpckhbw xmm12, xmm9
    pmullw    xmm12, %3
    paddw     xmm11, xmm12
%endmacro
```

مقادیر در ثبات‌های r10, r11, r12 برای دسترسی به سطرهای پیکسل‌های در تصویری که ورودی داده شده هستند (r10 مربوط به سطر که الان در آن هستیم می‌باشد، r11 مربوط به سطر قبلی (بالایی) و r12 هم مربوط به سطر بعدی (پایینی) هستند)، می‌باشند. در نتیجه در ماکرو MULL1، عدد متناظر در ماتریس را در مقدار پیکسلی که باید در آن ضرب کنیم انجام می‌دهیم. در این بخش به جای این که صرفاً روی 1 پیکسل عملیات انجام بدهیم، روی 16 پیکسل از تصویر به صورت همزمان انجام می‌دهیم. برای این که overflow در محاسبات به وجود نیاید 8 تا پیکسل رو با هم و 8 تا دیگر را جدا جدا انجام می‌دهیم که 8 تای اول را با استفاده از ثبات xmm13 و 8 تای دوم با استفاده از ثبات xmm12 انجام می‌شوند. حاصل نهایی هم در دو ثبات xmm10, xmm11 نگهداری می‌شوند. بعد از اتمام شدن عملیات‌ها، مقدار در ثبات rbx 16 اضافه می‌شود و سراغ 16 پیکسل بعدی می‌رویم.

3. در سطر با استفاده از در ثبات‌های r10, r11, r12، 16 تا 16 تا پیکسل‌های تصویر را در نظر می‌گیریم و روی آنها عملیات انجام می‌دهیم (برای پیکسل‌های ابتدایی و انتهایی عملیات انجام نمی‌دهیم چرا که 8 پیکسل در همسایگی خود ندارد). ممکن است تعدادی پیکسل در انتها باقی بمانند که نتوان آنها را دسته ای 16 تایی در نظر گرفت و روی آنها عملیات انجام داد (تعداد پیکسل‌های باقی مانده کمتر از 17 خواهند بود چرا که اگر کمتر از این مقدار باشد چون پیکسل انتهایی در عملیات‌ها شرکت نخواهد کرد و در نتیجه تعداد پیکسل‌هایی که می‌توان روی آنها عملیات انجام داد کمتر از 16 خواهد بود و دیگر برای عملیات موازی مطلوب نخواهند بود).

برای این دسته از پیکسل‌ها اینگونه عمل می‌کنیم:

```

MULL2 r11, -1, 0
MULL2 r11, 0, 4
MULL2 r11, 1, 8
MULL2 r10, -1, 12
MULL2 r10, 0, 16
MULL2 r10, 1, 20
MULL2 r12, -1, 24
MULL2 r12, 0, 28
MULL2 r12, 1, 32

```

```

%macro MULL2 3
    movzx ecx, byte [%1 + rbx + %2]
    imul ecx, dword [r8 + %3]
    add r14d, ecx
%endmacro

```

در این قسمت فقط روی یک پیکسل عملیات ضرب را انجام می‌دهیم (ثبات r8 مربوط به ماتریس می‌باشد) و حاصل را در ثبات r14 نگه می‌داریم. بعد از اتمام شدن عملیات‌ها، مقدار در ثبات rbx یکی اضافه می‌شود و سراغ پیکسل بعدی می‌رویم.

هنگامی که روی تمام پیکسل‌های یک سطر از تصویر عملیات انجام دادیم، مقدار در ثبات ebp یکی اضافه می‌شود (در ابتدا مقدار آن 1 می‌باشد و شماره سطری که در آن هستیم را نگه می‌دارد). و با استفاده از آن مقادیر در ثبات‌های r10, r11, r12 به روز می‌شوند و دوباره عملیات‌ها روی سطرهای جدید انجام می‌شوند.

نکته 1: این کار تا زمانی انجام می‌شود که روی تمام پیکسل‌های تصویر به جز پیکسل‌های در کادر تصویر (ستون اول و آخر و سطر اول و آخر) عملیات انجام شود. دلیل این که روی این پیکسل‌ها عملیات انجام نمی‌دهیم این می‌باشد که در همسایگی این پیکسل‌ها 8 تا پیکسل وجود ندارد و نمی‌توان convolution روی این پیکسل‌ها انجام داد. در تصویر خروجی پیکسل‌های حاشیه را مانند پیکسل‌های حاشیه در تصویر اصلی در نظر می‌گیریم.

نکته 2: مقادیر ذخیره شده برای خروجی همواره (در بخش 2 و 3) چک می‌شوند که مقدار آنها کمتر مساوی 255 و بزرگتر مساوی 0 باشند. اگر مقدار حاصل بیشتر از 255 بود، به جای آن مقدار 255 ذخیره خواهد شد و اگر مقدار حاصل کمتر از 0 بود، به جای آن مقدار 0 ذخیره خواهد شد. این کار به خاطر این می‌باشد که در تصویرها هر پیکسل مقداری بین 0 و 255 دارد و ما می‌خواهیم در انتها تصویر نهایی را نمایش دهیم.

در conv_c عملیات convolution با زبان سطح بالا C در تابع conv3x3 انجام می‌شود که دقیقاً کارهایی که در conv.asm انجام می‌شود، در این بخش هم انجام می‌شود.

حاصل در sum وجود دارد. ابتدا با استفاده از تابع clamp، مقدار آن در بازه بین 0 و 255 قرار می‌گیرد (مانند کاری که در conv.asm انجام دادیم) انجام می‌شود. و بعد در خروجی ذخیره می‌شود.

```

uint8_t clamp(int v) {
    if (v < 0) return 0;
    if (v > 255) return 255;
    return (uint8_t)v;
}

void conv3x3(const uint8_t* input, uint8_t* out, int w, int h) {
    // copy input -> out (keep borders)
    memcpy(out, input, (size_t)w * (size_t)h * 3);

    for (int y = 1; y < h - 1; y++) {
        for (int x = 1; x < w - 1; x++) {
            int idx = (y * w + x) * 3;

            for (int c = 0; c < 3; c++) {
                int sum = 0;

                sum += kernel[0][0] * input(((y - 1) * w + (x - 1)) * 3 + c);
                sum += kernel[0][1] * input(((y - 1) * w + x) * 3 + c);
                sum += kernel[0][2] * input(((y - 1) * w + (x + 1)) * 3 + c);

                sum += kernel[1][0] * input((y * w + (x - 1)) * 3 + c);
                sum += kernel[1][1] * input((y * w + x) * 3 + c);
                sum += kernel[1][2] * input((y * w + (x + 1)) * 3 + c);

                sum += kernel[2][0] * input(((y + 1) * w + (x - 1)) * 3 + c);
                sum += kernel[2][1] * input(((y + 1) * w + x) * 3 + c);
                sum += kernel[2][2] * input(((y + 1) * w + (x + 1)) * 3 + c);

                out[idx + c] = clamp(sum);
            }
        }
    }
}

```

در `conv_asm.c` کاری که قبل از انجام عملیات انجام می‌دهیم این هست که چون تصویر `rgb` می‌باشد ، ابتدا پیکسل‌های مربوط به هر رنگ را جدا می‌کنیم (در آرایه‌های `R1,G1,B1`) و بعد برای هر 3 دسته از پیکسل ها جداگانه (در مجموع 3 بار) عملیات `convolution` را انجام می‌دهیم و در انتها آرایه‌های حاصل (`R2,G2,B2`) را در کنار هم قرار می‌دهیم و تصویر نهایی را می‌سازیم. جدا کردن در تابع `planar_to_rgb` و کنار هم قرار دادن در تابع `rgb_to_planar` انجام می‌شود.

```

void rgb_to_planar(const uint8_t* rgb, uint8_t* R, uint8_t* G, uint8_t* B, int width, int height) {
    int n = width * height;
    for (int i = 0; i < n; i++) {
        R[i] = rgb[i*3 + 0];
        G[i] = rgb[i*3 + 1];
        B[i] = rgb[i*3 + 2];
    }
}

void planar_to_rgb(uint8_t* rgb, const uint8_t* R, const uint8_t* G, const uint8_t* B, int width, int height) {
    int n = width * height;
    for (int i = 0; i < n; i++) {
        rgb[i*3 + 0] = R[i];
        rgb[i*3 + 1] = G[i];
        rgb[i*3 + 2] = B[i];
    }
}

```

```

// make rgb separate
rgb_to_planar(image, R1, G1, B1, width, height);

// init output with input (so untouched border sta
memcpy(R2, R1, n);
memcpy(G2, G1, n);
memcpy(B2, B1, n);

double t1 = now_ms();

conv3x3(R1, R2, width, height, kernel);
conv3x3(G1, G2, width, height, kernel);
conv3x3(B1, B2, width, height, kernel);

double t2 = now_ms();

// make photo again
planar_to_rgb(out, R2, G2, B2, width, height);

```

برای اندازه گیری زمان در همه فایل ها از تابع `now_ms` استفاده می شود که با گرفتن زمان شروع و پایان عملیات ها زمان صرف شده برای انجام عملیات ها محاسبه می شود.

در `conv_total.c` توابع و دستورات جدیدی وجود ندارند و کارهایی که در `conv_c.c` و `conv_asm.c` انجام می شدند برای یک ورودی یکسان و در کنار هم انجام می شوند. (برای مقایسه زمان ها) و صرفاً 20 بار پشت سر هم انجام می شود تا زمان مورد نیاز به صورت میانگین بدست بیاید.

اجرا برنامه‌ها:

ابتدا برای تولید فایل‌های اجرایی، لازم است فایل‌های **object** مربوطه ساخته شوند و سپس عملیات لینک انجام گیرد. دستورات زیر به ترتیب اجرا می‌شوند:

- `nasm -f elf64 conv.asm -o conv.o`
- `gcc -c conv_c.c -o conv_c.o`
- `gcc -c conv_asm.c -o conv_asm.o`
- `gcc -c conv_total.c -o conv_total.o`
- `gcc conv_c.o -o conv_c -lm`
- `gcc conv_asm.o conv.o -o conv_asm -lm`
- `gcc conv_total.o conv.o -o conv_total -lm`

که در انتها 3 فایل اجرایی ساخته خواهد شد.

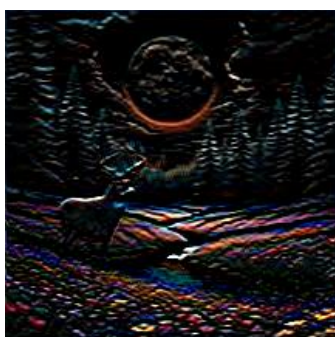
حال تصویر زیر را با نام **p1.jpg** در نظر می‌گیریم و با ماتریس convolution، $\begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ 1 & -1 & -1 \end{bmatrix}$ روی آن انجام می‌دهیم.



حال با دستور زیر ، تصویر حاصل از **conv_c** را با نام **p2.jpg** ذخیره می‌کنیم.

```
shahparsa@shahparsa:/mnt/c/Users/shahp/Desktop/Project$ ./conv_c p1.jpg p2.jpg 1 1 1 0 0 0 -1 -1 -1
C-only processing: 1.608 ms
C-only total processing: 22.560 ms
Done! Saved p2.jpg (216x215)
```

تصویر خروجی:



که زمان مورد نیاز برای عملیات convolution برابر 1.608 میلی ثانیه می باشد و زمان مورد نیاز برای کل برنامه برابر 22.560 میلی ثانیه می باشد.

حال با دستور زیر ، تصویر حاصل از conv_asm را با نام p3.jpg ذخیره می کنیم.

```
shahparsa@shahparsa:/mnt/c/Users/shahp/Desktop/Project$ ./conv_asm p1.jpg p3.jpg 1 1 1 0 0 0 -1 -1 -1
C+ASM processing: 0.050 ms
C+ASM total processing: 20.166 ms
Done! Saved p3.jpg (216x215)
```

تصویر خروجی:



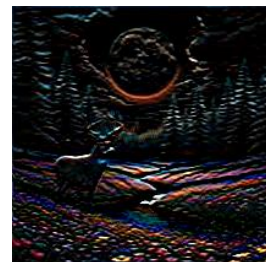
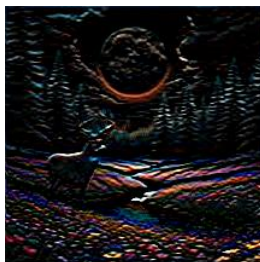
که زمان مورد نیاز برای عملیات convolution برابر 0.050 میلی ثانیه می باشد و زمان مورد نیاز برای کل برنامه برابر 20.166 میلی ثانیه می باشد.

حال با دستور زیر ، تصاویر حاصل از conv_total را با نام p4.jpg و p5.jpg ذخیره می کنیم.

```
shahparsa@shahparsa:/mnt/c/Users/shahp/Desktop/Project$ ./conv_total p1.jpg p4.jpg p5.jpg 1 1 1 0 0 0 -1 -1 -1
Results:
C-only time: 1.665 ms
C+ASM time: 0.045 ms
Speedup (C/ASM): 37.15x
```

در این برنامه 20 بار عملیات ها انجام شده است و طبق تصویر حدود سرعت برنامه با تابع asm ، 37 برابر می باشد. که این نشان می دهد استفاده از دستوراتی که به صورت موازی عملیات ها را انجام می دهند تا این حد می توانند روی زمان مورد نیاز برنامه ها تاثیر بگذارند.

تصاویر تولیدی هم برای هر دو مدل پردازش یکسان هستند.



قسمت 2 : فایل های برای این بخش:

moon_c.c / moon_asm.c / moon2_c.cpp / moon2_asm.cpp / blur.asm / conv_sobel.asm

در این بخش برنامه‌ای نوشته شده است که به آن تصویری دلخواه می‌دهیم و اگر در تصویر ماه کامل وجود داشت ، آن را شناسایی می‌کند و در تصویر خروجی دور آن خط قرمز می‌کشد.

ابتدا blur.asm را بررسی می‌کنیم. ساختار کلی مانند conv.asm که در بخش قبل بود ، می‌باشد و تنها تفاوتی که

وجود دارد این هست که ماتریسی که برای این پردازش استفاده می‌شود متفاوت است (ماتریس) $\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$ می‌باشد در انتها هم باید تقسیم بر 16 در هر عملیات انجام شود. و اعداد حاصل نیازی نیست بین 0 تا 255 باشند.

```
section .data
align 16
w1:  dw 1,1,1,1,1,1,1,1
w2:  dw 2,2,2,2,2,2,2,2
w4:  dw 4,4,4,4,4,4,4,4
w8:  dw 8,8,8,8,8,8,8,8
```

```
add  r14d, 8
shr  r14d, 4
```

```
movdqa xmm0, [rel w1]
movdqa xmm1, [rel w2]
movdqa xmm2, [rel w4]
movdqa xmm3, [rel w8]
```

```
; normalize: (sum + 8) >
paddw xmm10, xmm3
psrlw xmm10, 4
paddw xmm11, xmm3
psrlw xmm11, 4
```

دیگر LOAD_K وجود ندارد و از بخش .data ضرایب 1و2و4و8 را در ثبات‌های xmm0,...,xmm3 قرار می‌دهیم. برای تقسیم کردن هم ابتدا به 2 ثبات xmm10,xmm11 که حاصل عملیات‌ها در آنها قرار می‌گرفت ، 8 واحد اضافه می‌کنیم (برای تقریب بهتر) و بعد مقدار داخل آنها بر 16 تقسیم می‌کنیم. (با استفاده از دستورات شیفت) پس از آن مقادیر بعد از تقسیم ذخیره می‌شوند. برای پیکسل‌هایی هم که اضافه می‌مانند هم مانند شکل ابتدا 8 واحد به آنها اضافه می‌شود و سپس با استفاده از دستور شیفت ، تقسیم بر 16 می‌شوند.

به جای MULL2 در conv.asm هم MULL1,MULL2,MULL4 داریم که هر کدام عدد را در 1و2و4 ضرب می‌کنند.

```

%macro MULL1 2
    movzx ecx, byte [%1 + rbx + %2]
    add r14d, ecx
%endmacro

%macro MULL2 2
    movzx ecx, byte [%1 + rbx + %2]
    shl ecx, 1
    add r14d, ecx
%endmacro

%macro MULL4 2
    movzx ecx, byte [%1 + rbx + %2]
    shl ecx, 2
    add r14d, ecx
%endmacro

```

این تابع باعث می شود که نویزهای داخل تصویر از بین بروند (یا کمتر شوند) و این که لبه های داخل تصویر نرم تر شوند. حال `conv_sobel.asm` را بررسی می کنیم. ساختار کلی مانند `conv.asm` که در بخش قبل بود، می باشد و تنها تفاوتی که وجود دارد این هست که دیگر نیازی نیست اعداد حاصل از عملیات بین 0 تا 255 باشند و می توانند از $2^{15} - 1$ تا 2^{15} باشند. دیگر نیازی به چک کردن این که عدد حاصل چقدر هست نیست چرا که ضرایب در ماتریس `sobel` برابر 0 یا 1 یا 2 می باشند. همچنین خروجی به صورت 2 بایت، 2 بایت می باشد (s16) بر خلاف این که در دو فایل `asm` قبلی خروجی بایت، بایت بود.

```

; rdi = input image pointer (u8)
; rsi = output image pointer (int16)
; rdx = width
; rcx = height
; r8 = kernel pointer (9 * int32s)
conv3x3_u8_s16_asm:

```

و برای همین نحوه ذخیره در حافظه اینگونه می باشد: (ضریب 2 برای `rbx` به خاطر همین موضوع می باشد)

```

movdqu [r13 + rbx*2], xmm10
movdqu [r13 + rbx*2 + 16], xmm11

```

این تابع باعث می شود که تغییرات روشنایی در تصویر را (لبه) پیدا شوند. خود `sobel` یک ماتریسی هست که برای شناسایی لبه در تصویر استفاده می شود. و مدل های مختلفی بر اساس نوع لبه ای که می خواهیم پیدا کنیم دارد. ما دو

مدل $\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$ و $\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$ را استفاده می کنیم.

در همه فایل‌ها پردازش با ماتریس **blur** و دو نوع **sobel** گفته شده نیاز هست. یک مدل پردازش با استفاده از تابع‌های **asm** که گفته شده می‌باشد و نوع دیگر با استفاده از زبان سطح بالا **C** می‌باشد که توابع آنها به این شکل می‌باشند:

```
// 3x3 blur
void blur3x3(const uint8_t* in, uint8_t* out, int width, int height){
    for (int y = 1; y < height - 1; ++y) {
        for (int x = 1; x < width - 1; ++x) {
            int sum = 0;

            sum += in[(y-1)*width + (x-1)] * 1;
            sum += in[(y-1)*width + x] * 2;
            sum += in[(y-1)*width + (x+1)] * 1;
            sum += in[y*width + (x-1)] * 2;
            sum += in[y*width + x] * 4;
            sum += in[y*width + (x+1)] * 2;
            sum += in[(y+1)*width + (x-1)] * 1;
            sum += in[(y+1)*width + x] * 2;
            sum += in[(y+1)*width + (x+1)] * 1;

            out[y*width + x] = (uint8_t)(sum >> 4);
        }
    }
}
```

```
// 3x3 Convolution (u8 -> s16)
void conv3x3_u8_s16(const uint8_t* in, int16_t* out, int width, int height, const int kernel[9]){
    for (int y = 1; y < height-1; y++) {
        for (int x = 1; x < width-1; x++) {
            int sum = 0;

            sum += in[(y-1)*width + (x-1)] * kernel[0];
            sum += in[(y-1)*width + x] * kernel[1];
            sum += in[(y-1)*width + (x+1)] * kernel[2];
            sum += in[y*width + (x-1)] * kernel[3];
            sum += in[y*width + x] * kernel[4];
            sum += in[y*width + (x+1)] * kernel[5];
            sum += in[(y+1)*width + (x-1)] * kernel[6];
            sum += in[(y+1)*width + x] * kernel[7];
            sum += in[(y+1)*width + (x+1)] * kernel[8];

            out[y*width + x] = (int16_t)clamp(sum, -32768, 32767);
        }
    }
}
```

که همان کارهایی هست که در فایل‌های **asm** معادل آنها انجام می‌دادیم.

توابع در فایل‌های `conv_sobel.asm` و `blur.asm` اینگونه در فایل‌ها استفاده می‌شوند:

```
// ASM(Blur + Sobel)
extern "C" void blur3x3(const uint8_t* in, uint8_t* out, int width, int height);
extern "C" void conv3x3_u8_s16_asm(const uint8_t* in, int16_t* out, int w, int h, const int* k);
```

حال منطقی که باعث می‌شود در فایل‌ها بتوانیم تصویر ماه را تشخیص دهیم را تشریح می‌کنیم. (در تمام فایل‌ها مشترک است و صرفاً پردازش اولیه روی تصاویر با دو روش انجام می‌شود).

بعد از ورودی گرفتن تصویر ابتدا آن را از حالت `rgb` خارج می‌کنیم و آن را مانند تصویر با استفاده از فرمول `gray` می‌کنیم.

```
// RGB -> Gray
for (int i = 0; i < width*height; i++) {
    int r = img[i*3+0];
    int g = img[i*3+1];
    int b = img[i*3+2];
    gray[i] = (uint8_t)clamp((int)(0.299*r + 0.587*g + 0.114*b), 0, 255);
}
```

سپس روی تصویر `gray`، پردازش `blur` را انجام می‌دهیم. تا طبق گفته‌های قبلی نویز در تصویر کمتر شود و لبه‌های شدید، نرم‌تر شوند. در ابتدا هم همه خانه‌های در `blur` را 0 می‌کنیم که در نتیجه حاشیه تصویر حاصل سیاه خواهد بود. (بر خلاف کاری که داخل قسمت 1 انجام می‌دادیم)

```
// blur
memset(blur, 0, (size_t)width*height);
blur3x3(gray, blur, width, height);
```

سپس روی آرایه حاصل دو نوع متفاوت پردازش `sobel` را انجام می‌دهیم تا لبه‌های تصویر در جهت‌های مختلف پیدا شوند.

```
// Kernels Sobels
int Kx[9] = {
    -1,0,1,
    -2,0,2,
    -1,0,1
};

int Ky[9] = {
    -1,-2,-1,
    0,0,0,
    1,2,1
};
```

```
// sobel
memset(gx, 0, (size_t)width*height*sizeof(int16_t));
memset(gy, 0, (size_t)width*height*sizeof(int16_t));
conv3x3_u8_s16_asm(blur, gx, width, height, Kx);
conv3x3_u8_s16_asm(blur, gy, width, height, Ky);
```

مانند **blur**، آرایه‌های **gx** و **gy** در ابتدا همگی 0 در نظر گرفته می‌شوند که در نتیجه حاشیه‌ها همگی سیاه خواهند بود.

برای شناسایی ماه دنبال شی هستیم که هم نسبت به کل تصویر نور بیشتری داشته باشد و هم لبه باشد. برای همین در دو بخش زیر یک معیاری برای روشنایی بر اساس میانگین روشنایی کل تصویر محاسبه می‌کنیم. و بعد بر اساس این که هر پیکسل لبه است و هم از معیار روشنایی بیشتر هست، در آرایه **edge** به آن مقدار 1 یا 0 داده می‌شود. با این کار تعداد پیکسل‌هایی که نیاز هست چک کنیم کاسته می‌شود چرا که می‌دانیم برای ماه باید پیکسل‌های مرزی دو شرط گفته شده را داشته باشند.

```
// Mean brightness
long long sumB = 0;
for (int i = 0; i < width*height; i++) sumB += blur[i];
int meanB = (int)(sumB / (width*height));

int brightThreshold = meanB + 20;
if (brightThreshold > 230){
    brightThreshold = 230;
}
if (brightThreshold < 60){
    brightThreshold = 60;
}
```

```
int edgeThreshold = 180;

// Build edge mask: strong edge AND bright area
for (int i = 0; i < width*height; i++) {
    int ax = gx[i];
    if (ax < 0){
        ax = -ax;
    }
    int ay = gy[i];
    if (ay < 0){
        ay = -ay;
    }
    int mag = ax + ay;

    int isEdge = (mag > edgeThreshold);
    int isBright = (blur[i] > brightThreshold);

    if (mag > edgeThreshold && blur[i] > brightThreshold){
        edge[i] = 1;
    }else{
        edge[i] = 0;
    }
}
```

حال در ادامه، با استفاده از **Hough Transform** به دنبال یافتن یک شکل دایره‌ای در میان پیکسل‌های لبه استخراج‌شده می‌گردیم. در این روش، هر پیکسل لبه به‌عنوان یک رأی‌دهنده در فضای پارامترهای دایره در نظر گرفته می‌شود.

```
int bestScore = 0;
int bestX = 0, bestY = 0, bestR = 0;

int m;
if (width < height){
    m = width;
}else{
    m = height;
}

// و می‌توان عدد ها را تغییر داد
int r_min = m / 12;
int r_max = m / 2;

if (r_min < 8){
    r_min = 8;
}
if (r_max < 20){
    r_max = 20;
}
```

ابتدا یک بازه شعاع مطلوب برای دایره‌ای که می‌خواهیم پیدا کنیم در نظر می‌گیریم که این مقادیر می‌توانند تغییر کنند. در نتیجه این کار دایره‌های خیلی کوچک یا خیلی بزرگ دیگر در نظر گرفته نخواهند شد. (عموماً ماه در تصاویر در این ابعاد می‌باشد).

حال طبق تصویر، برای هر شعاعی برای ما مطلوب هست در بین پیکسل‌هایی که شرایط مطلوب ما را داشتند (**edge** آنها 1 بود)؛ برای هر پیکسل فرض می‌کنیم که این پیکسل می‌تواند روی محیط یک دایره با شعاع مشخص قرار داشته باشد. اگر این فرض درست باشد، باید بتوانیم یک مرکز دایره پیدا کنیم که این پیکسل روی محیط آن قرار بگیرد. برای این کار، زاویه‌های مختلف از 0 تا 360 درجه را بررسی می‌کنیم و با استفاده از روابط هندسی دایره، مختصات مرکز احتمالی را محاسبه می‌کنیم.

اگر مختصات مرکز به‌دست آمده داخل محدوده‌ی تصویر باشد، به آن نقطه یک رأی داده می‌شود. به این ترتیب، هر پیکسل لبه به چندین مرکز احتمالی رأی می‌دهد. اگر تعداد زیادی از پیکسل‌های لبه به یک نقطه‌ی خاص رأی بدهند، یعنی احتمال زیادی وجود دارد که آن نقطه مرکز یک دایره‌ی واقعی باشد.

```
// Hough vote
for (int r = r_min; r <= r_max; r += 2) {
    int* acc = (int*)calloc((size_t)width*height, sizeof(int));
    if (!acc) {
        printf("Out of memory (acc)\n");
        break;
    }

    for (int y = 0; y < height; y++) {
        for (int x = 0; x < width; x++) {
            // فقط نقاطی که لبه هستند بررسی می شود
            if (!edge[y*width + x]) continue;

            for (int t = 0; t < 360; t += 5) {
                float rad = (float)t * 3.1415926f / 180.0f;
                int cx = x - (int)lroundf(r * cosf(rad));
                int cy = y - (int)lroundf(r * sinf(rad));
                if ((unsigned)cx < (unsigned)width && (unsigned)cy < (unsigned)height)
                    acc[cy*width + cx]++;
            }
        }
    }

    for (int i = 0; i < width*height; i++) {
        if (acc[i] > bestScore) {
            bestScore = acc[i];
            bestY = i / width;
            bestX = i % width;
            bestR = r;
        }
    }

    free(acc);
}
}
```

سپس در بین نقاط گفته شده ، نقطه‌ای با بیشترین رای را در نظر می‌گیریم. اگر مقدار رای داده شده بیشتر از **acceptScore** باشد (مقدار این دست ما هست و براساس داده‌ها می‌توان مقدار مطلوب را پیدا کرد) می‌توان گفت که یک دایره‌ی معتبر در تصویر پیدا شده است. در پروژه‌ی ما، این دایره همان ماه خواهد بود. سپس دایره پیدا شده را در صورت وجود با خط قرمز مشخص می‌شود. (با استفاده از تابع **draw_circle**)

```
int acceptScore = 27;
if (bestScore > acceptScore) {
    printf("Moon found at (%d,%d) radius=%d score=%d\n",
           bestX, bestY, bestR, bestScore);
    draw_circle(img, width, height, bestX, bestY, bestR);
} else {
    printf("Moon not found (bestScore=%d)\n", bestScore);
}
```

```

void draw_circle(uint8_t* img, int width, int height, int cx, int cy, int r){
    int thickness = 1;

    for (int t = 0; t < 360; t++) {
        float rad = (float)t * 3.1415926f / 180.0f;
        int x0 = cx + (int)lroundf(r * cosf(rad));
        int y0 = cy + (int)lroundf(r * sinf(rad));

        for (int dy = -thickness; dy <= thickness; dy++) {
            for (int dx = -thickness; dx <= thickness; dx++) {
                int x = x0 + dx;
                int y = y0 + dy;
                if ((unsigned)x < (unsigned)width && (unsigned)y < (unsigned)height) {
                    int idx = (y*width + x) * 3;
                    img[idx+0] = 255;
                    img[idx+1] = 0;
                    img[idx+2] = 0;
                }
            }
        }
    }
}

```

در دو فایل `moon_c.c` و `moon_asm.c` برای یک تصویر ورودی عملیات‌ها انجام می‌شود و تنها تفاوت در پردازش اولیه تصویر می‌باشد. در دو فایل `moon2_asm.cpp` و `moon2_c.cpp` ساختار فایل‌های قبلی وجود دارد و صرفاً روی چندین عکس که در پوشه‌های `moon` و `no_moon` قرار دارند عملیات‌ها انجام می‌شود و در صورت وجود ماه در بین عکس‌های `moon` تصویر خروجی را در پوشه `output` قرار می‌گیرد.

اجرا برنامه‌ها:

ابتدا برای تولید فایل‌های اجرایی، لازم است فایل‌های `object` مربوطه ساخته شوند و سپس عملیات لینک انجام گیرد. دستورات زیر به ترتیب اجرا می‌شوند:

- `nasm -f elf64 blur.asm -o blur.o`
- `nasm -f elf64 conv_sobel.asm -o conv_sobel.o`
- `gcc -c moon_c.c -o moon_c.o`
- `gcc -c moon_asm.c -o moon_asm.o`
- `gcc moon_c.o -o moon_c -lm`
- `gcc moon_asm.o blur.o conv_sobel.o -lm -o moon_asm`
- `g++ -std=c++17 moon2_asm.cpp conv_sobel.o blur.o -o moon2_asm -lm`
- `g++ -std=c++17 moon2_c.cpp -o moon2_c -lm`

که در انتها 4 فایل اجرایی ساخته خواهد شد.

حال تصویر زیر را با نام p1.jpg در نظر می گیریم.



حال با دستور زیر ، تصویر حاصل از moon_c را با نام p21.jpg ذخیره می کنیم.

```
shahparsa@shahparsa:/mnt/c/Users/shahp/Desktop/Project$ ./moon_c p1.jpg p21.jpg
Image loaded: 216x215
Searching radius: r_min=17 r_max=107
Brightness mean=74 => brightThreshold=94, edgeThreshold=180
Moon found at (106,43) radius=31 score=54
C-only time: 142.054 ms
```

تصویر خروجی:



درنتیجه ماه را توانسته است پیدا کند و 142.054 میلی ثانیه طول کشیده است.

حال با دستور زیر ، تصویر حاصل از moon_asm را با نام p22.jpg ذخیره می کنیم.

```
shahparsa@shahparsa:/mnt/c/Users/shahp/Desktop/Project$ ./moon_asm p1.jpg p22.jpg
Image loaded: 216x215
Searching radius: r_min=17 r_max=107
Brightness mean=74 => brightThreshold=94, edgeThreshold=180
Moon found at (106,43) radius=31 score=54
C+ASM time: 140.917 ms
```

تصویر خروجی:



در نتیجه ماه را توانسته است پیدا کند و 140.917 میلی ثانیه طول کشیده است.

تفاوت زیادی از لحاظ زمانی دیده نمی شود چرا که بخشی که دایره را پیدا می کند از لحاظ زمانی زیاد طول می کشد و برای همین تاثیر استفاده از تابع های در asm زیاد دیده نمی شود.

حال با دستور زیر ، moon2_c را بر روی پوشه های گفته شده اجرا می کنیم.

```
shahparsa@shahparsa:/mnt/c/Users/shahp/Desktop/403110506$ ./moon2_c moon no_moon
[1] moon/moon1.jpg | actual=moon pred=moon | saved=output
[2] moon/moon10.jpg | actual=moon pred=moon | saved=output
[3] moon/moon11.jpg | actual=moon pred=no_moon
[4] moon/moon12.jpg | actual=moon pred=moon | saved=output
[5] moon/moon14.jpg | actual=moon pred=no_moon
[6] moon/moon15.jpg | actual=moon pred=moon | saved=output
```

نتایج:

```
===== RESULTS =====
TP (moon -> moon)      : 31
TN (no_moon -> no_moon) : 9
FP (no_moon -> moon)   : 1
FN (moon -> no_moon)   : 17
Total                  : 58
Accuracy               : 68.97%
Recall (TPR)          : 64.58%
Precision              : 96.88%
Specificity (TNR)     : 90.00%
F1-score              : 77.50%
Total time             : 25169.27 ms
```

دقت حدود 68 درصد می باشد و باید به این نکته دقت کرد که 49 تا عکسی که ماه دارند و 10 تا عکسی که در آنها ماه وجود ندارد ، در پوشه ها قرار دارند و برنامه تصاویری که در آنها ماه وجود ندارد را بهتر شناسایی می کند و اگر تعداد عکس هایی که در آنها ماه نباشد بیشتر بود ، دقت بالاتر می رفت.

حال با دستور زیر ، moon2_asm را بر روی پوشه های گفته شده اجرا می کنیم.

```
shahparsa@shahparsa: /mnt/c/Users/shahp/Desktop/403110506$ ./moon2_asm moon no_moon
[1] moon/moon1.jpg | actual=moon pred=moon | saved=output
[2] moon/moon10.jpg | actual=moon pred=moon | saved=output
[3] moon/moon11.jpg | actual=moon pred=no_moon
[4] moon/moon12.jpg | actual=moon pred=moon | saved=output
[5] moon/moon14.jpg | actual=moon pred=no_moon
[6] moon/moon15.jpg | actual=moon pred=moon | saved=output
```

نتایج :

```
===== RESULTS =====
TP (moon -> moon)      : 31
TN (no_moon -> no_moon) : 9
FP (no_moon -> moon)   : 1
FN (moon -> no_moon)   : 17
Total                  : 58
Accuracy               : 68.97%
Recall (TPR)           : 64.58%
Precision              : 96.88%
Specificity (TNR)      : 90.00%
F1-score               : 77.50%
Total time             : 23477.55 ms
```

نتایج مانند برنامه قبلی می باشد.

همان طور که دیده شد از لحاظ زمانی برنامه ای که با دستورات برداری asm زده شده بود ، بهتر می باشد ولی در مقابل مقدار زمانی که در کل مصرف شده است این تغییر زیاد نمی باشد که دلیل آن را در بالا توضیح دادیم. همچنین اگر معیارهایی که در برنامه در نظر گرفتیم برای حداقل امتیاز یا نور را اگر تغییر بدهیم ، ممکن هست دقت بیشتر شود.